

FoodETA：外卖配送时效与延迟因素可视分析系统

中期报告

成员 A 成员 B 成员 C

数据可视化导论课程项目

2026 年 5 月 30 日

摘要

FoodETA 面向外卖配送时效与延迟因素分析，基于 Kaggle 公开外卖配送数据，围绕配送时长、配送距离、天气、交通、订单时段、城市、骑手属性和车辆类型等因素，设计后续可扩展为多视图联动系统的数据分析方案。当前中期阶段已经完成数据理解、预处理脚本、距离计算工具、聚合 JSON 输出工具、分析任务定义和后续系统规划；React 前端、FastAPI 后端和交互式多视图界面属于后续工作，尚未在当前仓库中实现。

1 项目背景

外卖配送是即时物流中的典型场景。配送时效不仅受到配送距离影响，也会受到天气、交通密度、订单高峰、车辆类型、多单配送和骑手状态等因素共同影响。对于平台运营、配送调度和用户体验分析而言，仅观察平均配送时间不足以解释延迟订单的形成原因。

FoodETA 的目标是对外卖配送时效和延迟因素进行可视分析。项目关注的问题包括：哪些时段更容易出现延迟，天气和交通是否显著增加配送时长，短距离长耗时订单是否存在，骑手评分和车辆类型是否影响配送效率，以及不同城市或区域之间是否存在差异。当前阶段重点是建立可复现的数据处理流程和分析任务框架，为后续可视化系统实现提供稳定数据基础。

2 数据来源与规模

本项目使用 Kaggle 公开数据集 Food Delivery Dataset，数据集链接为：<https://www.kaggle.com/datasets/gauravmalik26/food-delivery-dataset>。

公开资料显示,该数据集约包含 45,000 条外卖配送记录,每条记录对应一次配送样本,字段覆盖配送时间、餐厅位置、配送位置、天气、交通、订单类型、车辆类型和骑手属性等信息。该规模满足课程建议“不少于 1 万条记录”的要求,适合用于多视图可视分析。原始数据不提交到仓库,使用者需要自行下载并放置为 data/raw/food_delivery.csv。

3 字段分类

数据字段可分为以下几类。

字段类别	说明
订单基础信息	包括 ID 和 Type_of_order, 用于识别订单样本并比较不同订单类型下的配送表现。清洗后对应 order_id 和 order_type。
配送员属性	包括 Delivery_person_ID、Delivery_person_Age 和 Delivery_person_Ratings, 用于分析骑手特征与配送表现之间的关系。清洗后对应 delivery_person_id、delivery_person_age 和 delivery_person_ratings。
空间位置属性	包括 Restaurant_latitude、Restaurant_longitude、Delivery_location_latitude、Delivery_location_longitude 和 City, 用于计算配送距离并比较城市差异。
时间属性	包括 Order_Date、Time_Orderd 和 Time_Order_picked, 用于分析订单日期、订单小时、取餐时间和时段差异。
环境与交通属性	包括 Weatherconditions、Road_traffic_density 和 Festival, 用于分析天气、交通和节日条件对配送效率的影响。
配送方式属性	包括 Type_of_vehicle、Vehicle_condition 和 multiple_deliveries, 用于比较不同车辆类型、车辆状态和多单配送情况。
配送结果变量	包括 Time_taken(min) 以及预处理生成的 delivery_duration_min、distance_km、is_delayed 和 speed_kmph, 用于衡量配送时效和延迟情况。

4 预处理流程

当前仓库已经包含三个数据处理脚本: `scripts/preprocess.py`、`scripts/compute_distance.py` 和 `scripts/aggregate_data.py`。

`scripts/preprocess.py` 是主入口, 运行方式如下:

```
python scripts/preprocess.py
```

该脚本读取 `data/raw/food_delivery.csv`, 将原始字段统一为 `snake_case`, 并进一步对齐到清洗后 `schema`。脚本兼容常见空格、大小写差异和 `NaN` 字符串; 从 `Time_taken(min)` 中提取配送时长数字; 当 `Time_Orderd` 缺失时尝试使用 `Time_Order_picked` 解析订单小时; 根据 `Order_Date` 解析星期。

`scripts/compute_distance.py` 实现经纬度合法性检查和 `haversine` 距离计算。预处理阶段会删除缺失或明显异常的配送时长、经纬度和距离记录, 并生成清洗后的 `data/processed/orders_clean.csv`。

`scripts/aggregate_data.py` 负责生成面向后续可视化的数据摘要, 输出包括 `overview_summary`、`delivery_time_distribution.json`、`distance_time_sample.json`、`time_period_summary.json`、`hour_summary.json`、`weather_traffic_summary.json`、`courier_vehicle_summary.json` 和 `city_summary.json`。

脚本运行时会打印原始记录数、清洗后记录数、删除记录数、主要字段缺失比例、延迟阈值和输出文件列表。若原始 CSV 不存在, 脚本会给出明确错误提示并以非零状态退出。

5 派生变量

预处理阶段生成以下主要派生变量:

- `delivery_duration_min`: 从 `Time_taken(min)` 中提取的配送时长, 单位为分钟;
- `distance_km`: 使用 `haversine` 公式计算的餐厅到配送地点距离, 单位为千米;
- `order_hour`: 订单小时, 优先来自 `Time_Orderd`, 缺失时尝试 `Time_Order_picked`;
- `weekday`: 由 `Order_Date` 解析得到的星期;
- `time_period`: 时段标签, 包括 `breakfast`、`lunch_peak`、`afternoon`、`dinner_peak`、`night` 和 `unknown`;
- `is_delayed`: 配送时长是否严格大于全体订单配送时长的 75 分位数阈值;
- `speed_kmph`: 近似配送速度, 异常速度置为空值。

6 分析任务

当前项目定义六个核心分析任务。

T1 配送时间整体分布

分析 `delivery_duration_min` 的中心趋势、离散程度和长尾延迟情况，判断整体配送效率是否稳定。

T2 配送距离与配送时间关系

基于 `distance_km` 和 `delivery_duration_min` 分析距离与时长的关系，识别短距离长耗时或长距离高效率的异常样本。

T3 天气与交通影响

比较 `weather` 和 `traffic_density` 组合下的平均配送时长、延迟率和订单量，判断外部环境压力。

T4 不同时段效率差异

基于 `order_hour`、`weekday` 和 `time_period` 分析早餐、午高峰、下午、晚高峰和夜间的订单量与配送效率差异。

T5 骑手属性与车辆类型影响

分析 `delivery_person_age`、`delivery_person_ratings`、`vehicle_type` 和 `multiple_deliveries` 对配送表现的影响。

T6 高延迟风险场景组合识别

通过 `time_period`、`weather`、`traffic_density`、`distance_km` 和 `multiple_deliveries` 等字段组合，识别延迟风险较高的典型场景。

7 初步可视化设计

后续系统计划采用多视图布局。Overview 视图展示总订单数、有效订单数、平均配送时长、延迟率、平均距离、天气类别数和交通类别数。配送时间分布视图使用直方图和箱线图展示时长分布。距离-时间散点图展示 `distance_km` 与 `delivery_duration_min` 的关系，并支持异常点筛选。时间视图展示小时和时段维度的订单量、平均时长和延迟率。天气交通视图使用热力矩阵或分组柱状图比较外部条件影响。骑手车辆视图按 `vehicle_type`、评分分箱和年龄分箱比较配送表现。城市视图用于展示不同城市或区域的配送差异。

交互设计将围绕筛选、brushing、图表点击联动和详情面板展开。当前阶段尚未实现前端界面，以上内容作为后续 React 可视化系统的设计依据。

8 经典设计借鉴与适配

可视分析系统设计需要借鉴已有研究中成熟的表达方式和交互思想，但本项目不会直接复用现有系统，而是结合外卖配送数据重新定义分析任务、视图组合和交互流程。当前阶段主要将 LineUp、Parallel Sets 和 TimeNotes 作为后续视图设计参考。

LineUp：多属性排序与配送风险场景

LineUp 的核心思想是支持多属性数据的排序、比较和权重组合分析。本项目计划借鉴这一思想，设计 Delivery Risk Ranking View，即配送延迟风险排序视图。该视图不直接罗列单条订单，而是将订单按条件组合聚合为配送风险场景，例如晚高峰 + 高交通密度 + 雨天、夜间 + 多单配送、远距离 + 高交通密度、低评分骑手 + 多单配送、节日 + 晚高峰等。

每个风险场景计划包含 `order_count`、`avg_delivery_duration_min`、`delay_rate`、`avg_distance_km`、`multiple_delivery_rate`、`avg_rating` 和 `risk_score` 等指标。中期阶段暂定的综合风险评分为：

```
risk_score =  
0.4 * normalized_avg_delivery_duration  
+ 0.3 * delay_rate  
+ 0.2 * normalized_avg_distance  
+ 0.1 * multiple_delivery_rate
```

该公式不是机器学习模型，而是用于可解释排序的可视分析指标，后续可以根据课程反馈调整权重。团队的设计贡献在于将 LineUp 的多属性排序思想适配到外卖配送场景，把天气、交通密度、时段、距离、多单配送和骑手评分组合成配送延迟风险场景，用于识别高延迟风险条件组合。

Parallel Sets：多类别路径与延迟因素组合

Parallel Sets 适合展示多个类别变量之间的流向关系和组合分布。本项目计划借鉴这一思想，设计 Delay Factor Flow View，即配送延迟因素流向图。该视图可展示 `weather -> traffic_density -> time_period -> is_delayed` 等路径，也可以扩展为 `city -> traffic_density -> time_period -> is_delayed` 或 `vehicle_type -> multiple_deliveries -> is_delayed`。

路径宽度表示订单数量，颜色表示延迟率或平均配送时长。该视图用于回答哪些天气和交通组合更容易导向延迟、晚高峰是否放大高交通密度影响、多单配送是否更容易与延迟共同出现，以及哪些订单量不大但延迟率较高的条件组合值得关注。团队的设计贡献在于将 Parallel Sets 的类别路径表达适配为外卖配送条件组合分析，相比单独柱状图更强调多因素组合对配送时效的影响。

TimeNotes：时间序列注释与配送时段解释

TimeNotes 强调在时间序列中标注重要事件、阶段或异常点，使用户能够结合上下文理解时间模式。本项目计划借鉴这一思想，增强 Temporal Pattern View，形成 Annotated Temporal Pattern View。

该视图将在小时趋势中标注 breakfast、lunch_peak、afternoon、dinner_peak 和 night 等业务时段，并标注订单量最高的小时、平均配送时长最高的小时、延迟率最高的小时以及异常峰值时段。示例注释包括“12:00 午高峰，订单量达到全天峰值”“19:00 晚高峰，平均配送时长较高”“22:00 夜间订单减少，但延迟率仍偏高”。团队的设计贡献在于把 TimeNotes 的时间注释思想适配到外卖配送时段分析，使时间趋势图不仅显示数值变化，也解释高峰和异常出现的业务语境。

总体而言，本项目自己的贡献不是简单更换数据集，而是围绕“发现问题、定位异常、解释原因、排序比较”的分析流程，将配送时间、距离、天气、交通、时段、骑手和车辆因素组织到统一的延迟风险分析框架中。

9 当前进展

截至中期阶段，项目已经完成以下内容：

- 明确项目主题和数据集来源；
- 建立 data/raw/ 与 data/processed/ 数据目录说明；
- 编写 requirements.txt，列出 pandas 和 numpy 等处理依赖；
- 实现 scripts/compute_distance.py，包括经纬度合法性检查和 haversine 距离计算；
- 实现 scripts/preprocess.py，包括字段标准化、配送时长解析、经纬度清洗、距离计算、时段派生、延迟标签和速度计算；
- 实现 scripts/aggregate_data.py，输出概览、分布、散点采样、时间、天气交通、骑手车辆和城市聚合 JSON；
- 完成数据说明、分析任务定义和后续工作规划文档。

当前本地已经放置并处理 Kaggle CSV，但原始数据和 processed 结果通过 .gitignore 避免提交到仓库。项目未创建完整前端或后端目录。

10 后续计划

后续将基于已经生成的清洗数据和聚合 JSON 设计初步图表截图或静态样例，并继续检查距离、时段和延迟阈值是否符合分析预期。随后计划使用 React + TypeScript + Vite 实现前端界面，使用 ECharts 和 D3 构建主要图表，并实现多视图联动、筛选器、异常点选择和详情面板。

如果静态 JSON 能满足课程演示需求，则优先保持前端直接读取 `data/processed/` 文件；如果需要动态筛选、分页或更大规模查询，再补充 FastAPI 服务层。计划接口包括 `/api/overview`、`/api/delivery-time-distribution`、`/api/distance-time-sample`、`/api/time-per`、`/api/weather-traffic-summary`、`/api/courier-vehicle-summary` 和 `/api/city-summary`。

11 三人分工

- 成员 A: 数据处理与字段解释。负责下载 Kaggle 数据、梳理字段含义、编写 `preprocess.py`、计算 `distance_km`、`time_period`、`is_delayed`、`speed_kmph`，生成 `processed` 数据，并在中期答辩中主讲数据来源和预处理流程。
- 成员 B: 分析任务与可视化设计。负责明确分析任务、设计后续视图、说明每个视图对应的分析问题、规划多视图联动，并在中期答辩中主讲分析任务和可视化设计理由。
- 成员 C: 文档、系统规划与后续前后端设计。负责整理 README、编写 `midterm_report.tex` 和 `future_work.md`、规划后续前端和后端架构，并在中期答辩中主讲系统规划、分工和后续工作。